MODERATED LEARNING MANAGEMENT SYSTEM

CS4433 Software Engineering — Individual Project Report

Technology: C++17 • CMake • SQLite • Argon2id • GitHub Actions

Contents

- Abstract
- 1. Introduction
- 2. Project planning and management
- 3. Requirements engineering
- 4. System design
- 5. Implementation
- 6. Verification and validation
- 7. Configuration management and quality assurance
- 8. Deployment
- 9. Maintenance and Change Request CR1
- 10. Professional, legal and ethical considerations
- 11. Conclusion
- Appendix A. Diagram and image inventory
- Appendix B. Requirements-to-evidence summary
- Appendix C. Key project references

Abstract

The Moderated Learning Management System (LMS) is an individual software-engineering project for a secondary-school environment. It provides a local/offline C++17 console application for account and course administration, assignment creation, student submissions, marking, moderation and controlled release of results. SQLite provides persistence, Argon2id is used for password hashing, and GitHub Actions supports automated build-and-test verification.

This report consolidates the project life cycle and available evidence from planning, requirements, design, implementation, verification, configuration management, deployment preparation and maintenance. It distinguishes implemented and tested behaviour from items that still require manual confirmation. The project has a successful recorded Windows/MSYS2 build, 12/12 passing registered CTest tests, 82% line coverage across seven measured source files, and a final Cppcheck run with zero diagnostics in the enabled categories. These results are meaningful but do not prove that every interactive acceptance workflow has been completed.

1. Introduction

The case study concerns a secondary school that needs to manage courses, assignments and assessment for four roles: Student, Lecturer, Head of Department (HoD) and Administrator. The system must enforce role-specific permissions, preserve assessment history, and prevent students from viewing marks before HoD approval and release.

The implemented scope is a C++17 console application intended for one school computer or a local network and able to operate without reliable internet access. Online payments, video conferencing, automated grading, plagiarism detection, and web/mobile interfaces are outside the assignment scope.

The principal engineering objective is to translate stakeholder needs into a traceable design and a persistent, testable application. The project follows a Waterfall development structure, with V-Model verification and validation activities.

2. Project planning and management

The assignment allocates seven weeks to the project. The prescribed sequence is planning and management (week 1), requirements engineering (weeks 1–2), system design (weeks 2–3), implementation (weeks 4–5), verification and validation (weeks 5–6), deployment (week 6), and maintenance and completion (week 7). Phase gates are intended to review requirements, design and verification before progressing.

The planning artefact set is defined to include a feasibility study, work breakdown structure, activity dependencies, milestones, a Gantt plan, critical-path planning and a risk register. The repository's Phase 1 README lists these items, but the material inspected for this report did not provide verifiable detailed schedule values or a completed risk register. Those details should be checked against the original Phase 1 deliverable before submission; no dates, owners or risk scores are invented here.

Feasibility is supported by the selected technologies: C++17, CMake and SQLite are suitable for a local application; the MSYS2 UCRT64 toolchain was used successfully on Windows. Operational feasibility depends on school staff being able to use the console menus and maintain the local database. Economic feasibility benefits from open-source tools, while legal feasibility requires appropriate handling of learner information and assessment records. Schedule feasibility is constrained by the seven-week assignment and the need to complete manual acceptance checks.

3. Requirements engineering

Stakeholders identified in the project include the Principal, Lecturer, HoD, IT Officer, Student representative and Administrator. The four system actors are Student, Lecturer, HoD and Administrator. Their principal interests are usability, controlled mark release, deadline flexibility, fair late penalties, written feedback, offline operation, password protection and adequate response time for a school population of approximately 800 students.

Key conflicts were resolved in favour of controlled release: the lecturer's preference for immediate student access is constrained by the Principal's and HoD's moderation requirement. Students may resubmit before the effective deadline

and receive written feedback. Late work is governed by a consistent elapsed-time policy, and extensions are available to individuals and, after CR1, an entire class.

The SRS and use-case artefacts identify three central use cases: Submit Assignment, Mark Submission, and Moderate and Release Marks. The four actors and role-based workflows are reflected in the implemented application. The complete Phase 2 PDF remains the detailed source for requirement identifiers, priorities and fully dressed use-case flows.

Requirements traceability is maintained in `Phase-4-Implementation/docs/Traceability-Matrix.md`. It links requirements to design elements, implementation files and verification suites. It also explicitly marks areas where further case-specific or manual verification is required.

4. System design

The system uses a four-layer architecture: (1) presentation through console menus, (2) business logic for authentication, authorization, assignment, submission, penalties, marking, moderation, release and audit, (3) a repository/data-access layer, and (4) SQLite persistence. This separation keeps SQL and persistence concerns out of the user-interface workflow and supports focused testing.

The design applies the Strategy pattern to late-penalty calculation and the Repository pattern to data access. The Strategy abstraction permits the penalty policy to change without rewriting the submission workflow. Repository interfaces isolate business logic from SQLite-specific queries. The design rationale also discusses SOLID principles, particularly single responsibility and dependency inversion.

The domain model includes User and role-specific Student, Lecturer, HeadOfDepartment and Administrator classes, together with Department, Course, Enrolment, Assignment, DeadlineExtension, Submission, Mark, ModerationRecord and AuditRecord. Behavioural models cover the submission lifecycle, end-to-end activity, submission sequence, and moderation/release sequence.

The relational schema contains User, Department, Student, Lecturer, HeadOfDepartment, Course, CourseLecturer, Enrolment, Assignment, DeadlineExtension, Submission, Mark, ModerationRecord and AuditRecord tables. Many-to-many relationships use linking tables. Foreign keys, uniqueness constraints, checks and indexes support integrity. AuditRecord has database triggers that reject ordinary UPDATE and DELETE operations.

Supplied diagrams are maintained separately in the `images/` folder rather than embedded in the report. The combined UML image is a reference artefact; individual diagrams are also provided where available. The diagram inventory is listed in Appendix A.

5. Implementation

The application is written in C++17 and built with CMake. The verified Windows environment uses MSYS2 UCRT64, GCC, SQLite and Argon2; Windows BCrypt supplies secure salt generation. The application uses a local `lms.db` file in the project root, so it should be launched from that directory. The repository intentionally excludes a production database file.

Authentication stores salted password hashes rather than plaintext passwords. Data access is performed through repository classes and SQLite prepared/parameterised statements. Role-specific workflows support administration, assignment creation, submission, marking, moderation and mark release. Database constraints and application validation are used together to protect persistent data.

Submission attempts are stored separately. Attempt allocation and insertion use a `BEGIN IMMEDIATE` transaction. Submission timestamps are captured once and reused for validation and persistence. Deadlines are normalised to `YYYY-MM-DD HH:MM:SS`; date-only legacy values are treated as 23:59:59. Individual and whole-class extensions are represented by DeadlineExtension records, with a null student identifier representing a class-wide extension.

The implemented late penalty is 10% of the assignment maximum mark for each completed 24-hour period, up to 72 hours. New first submissions more than 72 hours late are rejected; resubmissions are restricted to the effective deadline. This follows the specific exception in the Submit Assignment use case, although earlier business-rule

wording says late work after three days receives zero. The inconsistency should remain explicitly acknowledged unless the approved SRS is formally reconciled.

Marking and moderation support written feedback, correction requests, approval, release and audit history. The database schema constrains mark values to be non-negative, but the schema's CHECK constraints alone do not impose the assignment maximum; application-level validation is therefore important.

6. Verification and validation

The test strategy uses unit and integration tests as part of a V-Model approach. The recorded Windows build completed successfully and the latest documented CTest run passed all 12 registered tests. The registered suites cover late-penalty boundaries, repository integration, submission workflow, mark/moderation/release and database behaviour.

The recorded gcovr report measured 1,072 lines across seven selected source files, of which 880 were executed, giving 82% overall line coverage. Individual results were: AdministrationRepository.cpp 90%, Database.cpp 83%, DeadlineExtensionRepository.cpp 76%, MarkingRepository.cpp 68%, ModerationRepository.cpp 86%, SubmissionRepository.cpp 76%, and DeadlineCalculator.cpp 100%. The 82% figure applies only to those seven files, not the entire application. MarkingRepository.cpp is below the 70% target and remains an improvement area.

> Phase 6 static analysis used Cppcheck 2.21.0 on 17 source files. The initial run reported 32 findings: 27 uninitialised-member warnings and five return-by-reference performance suggestions. Numeric members were given default initialisers and selected string getters in User and Assignment were changed to return const references. The final recorded output contains no diagnostics in the enabled warning, performance and portability categories.

Code metrics collected with scc 4.1.0 for `src/` and `include/` report 34 files, 4,934 total lines, 4,039 code lines and an aggregate complexity score of 604. This tool-reported aggregate must not be interpreted as average cyclomatic complexity per function. The focused self-review covered the eight modified files; it was not a complete manual review of every source file.

The test plan and traceability matrix identify outstanding manual verification: complete interactive workflows for all four roles, selected authorization boundaries, concurrency behaviour, and confirmation of individual audit assertions. The defect log contains no verified defect entry; it must not be interpreted as proof that no defects occurred.

7. Configuration management and quality assurance

The project uses Git branches to separate phase work. The `main` branch contains the integrated baseline, with separate Phase 5 and Phase 6 branches. Phase 5 was merged to main through pull request review. Phase 6 configuration management and QA work is now present on the `phase-6-configuration-management-qa` branch.

GitHub Actions is configured to run on branch pushes and pull requests to main. The workflow checks out the repository, provisions MSYS2 UCRT64 dependencies, configures with CMake, builds the application and tests, and runs CTest. The recorded workflow definition is available at `.github/workflows/ci.yml`.

Phase 6 evidence includes a documented branching and commit strategy, Cppcheck initial/final outputs, a code metrics report and a focused self-code-review record. Meaningful commits, pull requests, CI checks and semantic version tags are the intended configuration controls. A remote release tag was not independently confirmed during this report preparation.

8. Deployment

The target deployment is a Windows school computer or local network workstation running the console application and a local SQLite database. The verified build environment is Windows with MSYS2 UCRT64, GCC, CMake, SQLite and Argon2. The application also links Windows BCrypt.

Installation requires the source repository, a compatible toolchain and the required libraries. Configure from the project root using CMake with the MinGW Makefiles generator and the MSYS2 UCRT64 prefix, build the target, run CTest, and launch `lms.exe` from the project root. The first run prompts creation of an Administrator if the database has none. Subsequent account management is restricted to an Administrator.

The root-level database is persistent application data. It must be backed up before upgrading or running a version that may migrate the schema. The repository README notes that the source package does not include the database and that the application creates missing tables and applies a backward-compatible `is_active` migration.

The assignment explicitly requires a UML Deployment Diagram, installation guide, role-specific user manuals and v1.0 release notes. The supplied ZIP contains nine diagram/image files but no clearly identifiable deployment diagram. A deployment diagram and polished technician/role manuals remain outstanding evidence and should be completed before final submission. This report does not mislabel an architecture or component diagram as a deployment diagram.

9. Maintenance and Change Request CR1

CR1 states: “A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list.” This is an adaptive/perfective functional enhancement to the existing deadline-extension capability.

Impact analysis identifies the Lecturer workflow, Student assignment-list workflow, DeadlineExtension model/repository, effective-deadline resolution, assignment display, database representation and regression tests as affected. The existing schema supports both scopes by allowing `DeadlineExtension.student_id` to be nullable: a student identifier targets an individual, while a null student identifier represents a whole-class extension. The Phase 4 implementation documentation records student ID `0` as the lecturer workflow input for a whole-class extension.

Regression evidence includes the repository integration and submission workflow suites, and the traceability matrix maps individual and whole-class extensions to `DeadlineExtensionRepository.cpp`, `main.cpp` and the relevant tests. The Phase 4 README also records whole-class extensions as implemented. However, the precise UI assertion that the class extension is visibly displayed on every student's assignment list should be manually demonstrated and captured before claiming complete acceptance of the second half of CR1.

The CMake project version is 1.1, and the implementation includes CR1-related functionality. A Git tag named v1.1 was not independently confirmed in the inspected repository state. The report therefore identifies the codebase as the v1.1 maintenance implementation but does not claim a published v1.1 tag without evidence.

10. Professional, legal and ethical considerations

The LMS processes learner identities, account credentials, enrolments, submissions, marks, feedback and moderation history. These are sensitive educational records. The system should collect only information needed for its educational purpose, restrict access by role, protect credentials with salted password hashing, secure database backups, limit physical access to the host computer, and establish retention and deletion procedures with the school. Applicable Lesotho data-protection requirements and institutional policies must be checked by the school before deployment; this report does not claim formal legal compliance.

Assessment integrity requires that marks are not visible before HoD release, that moderation decisions are attributable, and that mark changes preserve old/new values, actor, time and reason where applicable. Append-only audit triggers provide a technical safeguard against ordinary update/delete operations, but administrators must also protect the database file and backups.

Fairness requires a transparent and consistently applied late policy, clear communication of effective deadlines and extensions, and a process for exceptional circumstances. The 10%-per-completed-24-hour penalty is based on the assignment maximum mark. The unresolved conflict between zero-after-72-hours wording and rejection of new submissions after 72 hours must be formally settled and documented so that students are treated consistently.

Accessibility is a limitation of the console-only interface. Keyboard-driven menus can support some users, but the project does not provide evidence of conformance to an accessibility standard, screen-reader testing, scalable text or

alternative interfaces. The school should evaluate the interface with users with disabilities and provide reasonable accommodations or an accessible alternative where required.

Professional responsibilities include honest reporting of test coverage and known gaps, protecting learner records, documenting assumptions, avoiding unsupported claims of security or legal compliance, and ensuring that users understand the consequences of deadline and mark decisions.

11. Conclusion

The project has produced a local/offline C++17 moderated LMS with SQLite persistence, role-specific workflows, submission and extension handling, marking, moderation, controlled mark release and audit records. The design uses layered architecture, Strategy and Repository patterns, and a relational database with integrity constraints.

Available evidence demonstrates a successful recorded build, 12/12 passing automated tests, 82% coverage over seven selected files, and zero final Cppcheck diagnostics in the enabled categories. The Phase 6 branch is present in GitHub. Important limitations remain: complete manual acceptance testing, verification of student-visible class extensions, the deployment diagram, polished installation and user manuals, release-tag confirmation, and final resolution of the late-submission policy wording.

Appendix A. Diagram and image inventory

All supplied diagrams are stored as separate files in the `images/` folder. The report references them externally rather than embedding them. The image filenames are retained from the supplied ZIP.

Image file	Purpose / diagram type
Activity Diagram With SwimLanes.png	End-to-end activity and responsibilities
Combined_UML_diagram.png	Combined UML overview
Component Diagram.png	System component architecture
ERD diagram.png	Relational database entity-relationship model
Finite State Machine diagram.png	Submission lifecycle state machine
Sequence Diagram_Moderate and Release Marks.png	Mark moderation and release sequence
Sequence Diagram_Submit Assignment.png	Assignment submission sequence
Use Case Diagram.png	Use-case model and actors
diagram.png	Additional supplied diagram; inspect against the original phase artefact before assigning a formal title

Appendix B. Requirements-to-evidence summary

FR: role-based administration, assignment creation, submissions/resubmissions, extensions, marking, moderation, controlled release and audit. Design: use-case/class/component/activity/sequence/state-machine/ERD artefacts. Implementation: `src/`, `include/`, `database/schema.sql`. Verification: CTest suites and the traceability matrix. Manual confirmation remains required for full interactive workflows and selected authorization/audit cases.

Requirement area	Design / implementation	Verification status
Account and role permissions	User / role classes; UserRepository and role workflows	Manual role/authorization checks still required
Submit and resubmit work	Submission model and sequence; SubmissionRepository	submission_workflow; exact manual exception checks remain
Late penalties and extensions	Strategy; DeadlineExtension; DeadlineCalculator	late_penalty_boundaries; repository_integration
Mark, moderate and release	Marking/Moderation services and	mark_moderation_release; HoD

	sequences	authorization checks need confirmation
Audit mark changes	AuditRecord and immutable triggers	Automated audit assertions plus manual record inspection
Whole-class extension visibility	DeadlineExtension and student assignment display	Automated extension coverage; manually demonstrate assignment-list visibility

Appendix C. Key project references

Repository main branch: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/tree/main

Phase 6 QA branch: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/tree/phase-6-configuration-management-qa

CI workflow: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/blob/main/.github/workflows/ci.yml

Database schema: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/blob/main/database/schema.sql

Phase 4 traceability matrix: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/blob/main/Phase-4-Implementation/docs/Traceability-Matrix.md

Phase 6 static analysis report: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/tree/phase-6-configuration-management-qa/blob/phase-6-configuration-management-qa/Phase-6-Configuration-Management-QA/docs/Static-Analysis-Report.md

Phase 6 code metrics report: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/tree/phase-6-configuration-management-qa/blob/phase-6-configuration-management-qa/Phase-6-Configuration-Management-QA/docs/Code-Metrics-Report.md

Phase 6 code review record: https://github.com/Kobo-Ramosoeu/CS4433-Moderated-LMS/tree/phase-6-configuration-management-qa/blob/phase-6-configuration-management-qa/Phase-6-Configuration-Management-QA/docs/Code-Review-Record.md